

Architecture Documentation

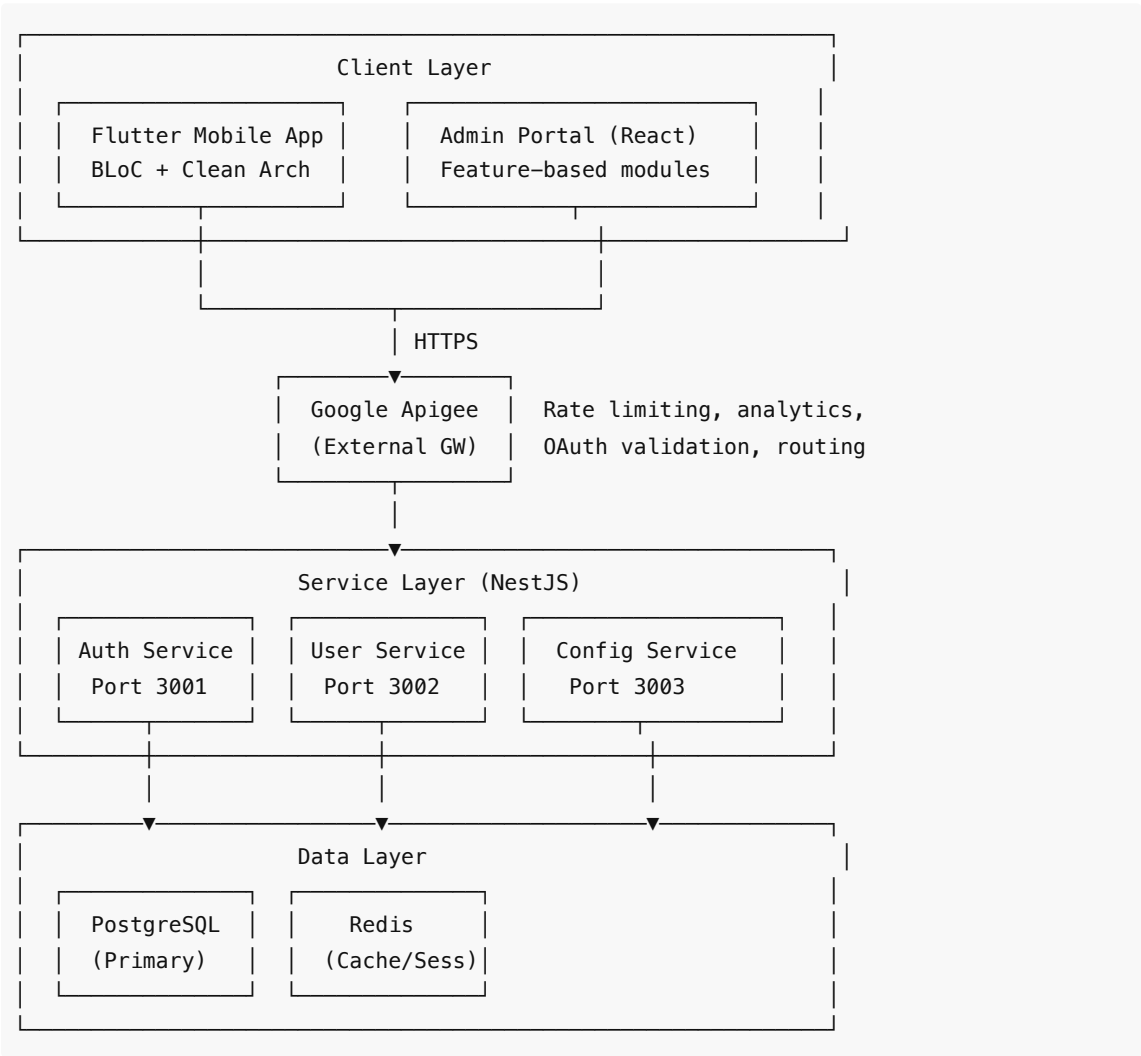
1. System Context

The platform serves two user groups:

User Group	Application	Access
Employees	Flutter Mobile App	Mobile only
Administrators	Web Admin Portal	Browser

All client applications communicate with backend microservices through **Google Apigee** (external API gateway, configured in a later phase).

2. High-Level Architecture



3. Backend — Microservices

3.1 Service Boundaries

Service	Responsibility	Port
auth-service	Authentication, JWT, OAuth2, demo 2FA	3001
user-service	User CRUD, profiles, roles	3002
config-service	App configuration, business settings, squad limits	3003

Each service is:

- Independently deployable
- Independently testable
- Owns its database schema (shared PostgreSQL instance, separate schemas)

3.2 Internal Architecture (per service)

```
src/
├─ main.ts                # Bootstrap
├─ app.module.ts          # Root module
├─ config/                # Environment configuration
├─ modules/
│   └─ <feature>/
│       ├── <feature>.module.ts
│       ├── <feature>.controller.ts    # Presentation
│       ├── <feature>.service.ts       # Business logic
│       ├── <feature>.repository.ts    # Data access
│       ├── dto/                      # Data transfer objects
│       └─ entities/                  # Database entities
├─ common/                 # Filters, guards, interceptors, pipes
└─ infrastructure/        # Database, external integrations
```

3.3 Cross-Cutting Concerns

Concern	Implementation
Authentication	JWT guards, OAuth2 strategy (Passport)
Authorization	Role-based guards
Validation	class-validator DTOs
Logging	Structured JSON logging
Health checks	@nestjs/terminus
API docs	Swagger/OpenAPI per service
Error handling	Global exception filter

3.4 Demo 2FA — Replaceable Design

The demo 2FA is implemented behind an abstraction:

```
ITwoFactorProvider (interface)
├─ DemoTwoFactorProvider   ← Current (temporary)
└─ ProductionTwoFactorProvider ← Future (swap via DI)
```

Replacing demo 2FA requires only:

1. Implement `ITwoFactorProvider`
2. Update DI binding in `AuthModule`
3. Update environment configuration

4. Flutter Mobile App

4.1 Layer Architecture

```
lib/
├─ app/                # App widget, router, theme
├─ core/               # Shared infrastructure
│   ├── config/        # Environment config
│   ├── di/            # Dependency injection (get_it + injectable)
│   ├── network/       # Dio client, interceptors
│   ├── storage/       # Secure storage wrapper
│   ├── localization/  # Phrase integration, l10n
│   ├── theme/         # App theme, RTL support
│   └─ error/          # Failure types, exceptions
├─ features/           # Feature modules
│   └─ <feature>/
│       ├── data/
│       │   ├── datasources/ # Remote & local data sources
│       │   ├── models/      # DTOs (freezed + json_serializable)
│       │   └─ repositories/ # Repository implementations
│       ├── domain/
│       │   ├── entities/    # Business entities
│       │   ├── repositories/ # Repository interfaces
│       │   └─ usecases/     # Business use cases
│       └─ presentation/
│           ├── bloc/        # BLoC (events, states)
│           ├── pages/       # Screens
│           └─ widgets/      # Feature-specific widgets
└─ main.dart
```

4.2 State Management

- **BLoC** (`flutter_bloc`) for all feature state
- **Repository pattern** for data access
- **Use cases** for business logic orchestration
- **Dependency injection** via `get_it` + `injectable`

4.3 Localization

- **Phrase Strings** for translation management
- Generated `l10n` files via `flutter gen-l10n` + Phrase CLI
- RTL/LTR automatic switching based on locale

- No hardcoded user-facing strings

5. Admin Portal

5.1 Architecture

```
src/
├─ api/           # API client (Axios), service modules
├─ components/    # Shared UI components
├─ features/      # Feature modules
│   ├── auth/
│   ├── users/
│   └─ configuration/
├─ hooks/         # Custom React hooks
├─ layouts/       # Page layouts
├─ pages/         # Route pages
├─ config/        # Environment configuration
├─ router/        # React Router setup
├─ store/         # State management (if needed)
└─ utils/        # Utilities
```

5.2 Tech Stack

- React 18 + TypeScript
- Vite (build tool)
- React Router (routing)
- Axios (HTTP client)
- Tailwind CSS (styling)

6. Security Architecture

Layer	Mechanism
Transport	TLS 1.2+ (HTTPS everywhere)
Authentication	JWT (access + refresh tokens)
Authorization	Role-based access control (RBAC)
Token storage (mobile)	Flutter Secure Storage
Token storage (web)	HttpOnly secure cookies
API security	Input validation, rate limiting (Apigee), CORS
Mobile security	Certificate pinning (configurable), root/jailbreak detection (future)

7. Environment Strategy

Config	Development	Demo	UAT	Production
Debug logging	Yes	Yes	Limited	No

Demo 2FA	Yes	Yes	No	No
Swagger UI	Yes	Yes	Yes	No
Hot reload	Yes	No	No	No

Configuration is loaded from environment variables — never hardcoded.

8. CI/CD Pipeline

See [docs/cicd/CI CD.md](#) for full pipeline details.

Each application has independent pipelines:

- Lint → Test → Build → Deploy (Demo)

9. Reporting Readiness

Backend models include:

- Audit timestamps (`createdAt` , `updatedAt`)
- Soft delete support where applicable
- Structured entities suitable for Power BI export
- Dedicated reporting endpoints (to be defined with business requirements)

10. Apigee Readiness

Services expose:

- Standard REST endpoints with versioning (`/api/v1/...`)
- Health check endpoints (`/health`)
- OpenAPI/Swagger specs
- Stateless design (JWT, no server-side sessions)
- Consistent error response format

Apigee will handle: routing, rate limiting, analytics, API key management, OAuth validation.

This document will be updated as business requirements are finalized.